UNIVERSIDAD DE ANTIOQUIA
Facultad de Ingeniería

Informática II
Desafío II
Octubre 9 de 2026
Alison Valencia García - 1025888980
Fénix Pamela Quintero Rodríguez - 1031814395

PREINFORME – DESAFÍO II

Ude@Book – Análisis y diseño de la solución

1. Análisis del problema

El principal reto de Ude@Book no es solamente conseguir que funcionen las opciones de una red social, sino organizar la información de manera que podamos realizar las operaciones utilizando la menor cantidad de recursos posible.

Uno de los problemas que identificamos es cómo representar las relaciones entre usuarios. Una persona puede tener muchos amigos y solicitudes pendientes, pero no tendría sentido guardar nuevamente toda la información de cada usuario dentro de esas relaciones. Esto generaría datos repetidos y aumentaría el consumo de memoria. También debemos considerar que algunas funcionalidades, como las sugerencias de amistad, pueden necesitar bastantes comparaciones. Si por cada posible sugerencia recorremos innecesariamente toda la información, la cantidad de operaciones puede crecer demasiado cuando aumente el número de usuarios.Algo similar ocurre con los mensajes y las fotografías. Necesitamos organizar los mensajes para facilitar su consulta por fechas, mientras que para las fotografías basta con manejar sus rutas y datos, sin cargar las imágenes completas.

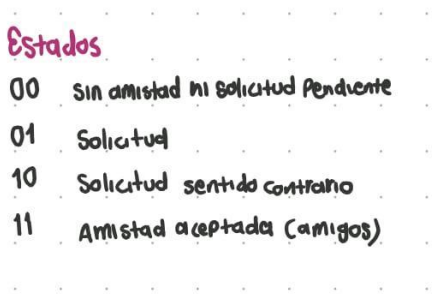
Por eso, nuestra propuesta se concentra en tres aspectos: evitar información duplicada, reducir recorridos innecesarios y escoger estructuras que permitan consultar los datos rápidamente sin consumir demasiada memoria. También tendremos en cuenta que debemos medir las iteraciones y la memoria utilizada, por lo que estas decisiones tendrán que comprobarse durante la implementación.

2. Consideraciones para la alternativa de solución propuesta

- Matriz de adyacencia optimizada

Para manejar las amistades y solicitudes proponemos utilizar una sola matriz de adyacencia. Sin embargo, en lugar de usar una matriz convencional de enteros, queremos aprovechar las operaciones a nivel de bits para consumir menos memoria.

La idea es representar cada relación entre dos usuarios utilizando únicamente 2 bits, con cuatro estados:



Esto nos permitiría comprobar si dos usuarios son amigos, si ya existe una solicitud y quién la envió, consultando una sola posición lógica.

Además, como las amistades son mutuas y solo puede existir una solicitud pendiente entre dos personas, no necesitamos guardar la información dos veces. Por eso proponemos utilizar una matriz triangular, sin diagonal, donde cada pareja de usuarios tenga un único estado. Los datos se almacenarían en un arreglo dinámico de unsigned char, aprovechando sus bits individualmente. Asumiendo bytes de 8 bits, podríamos guardar cuatro relaciones en cada byte.

Por ejemplo, para 1.000 usuarios existen 499.500 parejas posibles. Con 2 bits por pareja necesitaríamos 124.875 bytes, aproximadamente 122 KiB, sin contar los gastos adicionales del arreglo. Esto es mucho menos que utilizar matrices convencionales de enteros.

La principal ventaja es que podemos consultar o modificar una relación en tiempo constante, O(1), una vez conocidos los índices de los usuarios. Así evitamos recorrer listas para saber si alguien ya envió una solicitud. La dificultad está en calcular correctamente la posición de cada pareja y manipular los bits. Aun así, consideramos que vale la pena por el ahorro de memoria y la rapidez de las consultas.

- Sugerencias de amistad

Para las sugerencias aprovecharemos la misma matriz, sin crear otra estructura permanente con los amigos recomendados.

Podemos calcular los amigos en común comparando las relaciones de dos usuarios. Sin embargo, revisar todos los candidatos puede generar bastantes iteraciones, así que proponemos conservar temporalmente solo las mejores K sugerencias solicitadas.

Como K estará entre 1 y 20, no necesitamos almacenar y ordenar todos los posibles candidatos. Podemos mantener una pequeña colección con los mejores resultados encontrados e ir reemplazándolos cuando aparezca alguien con más amigos en común.

Esto reduce la memoria auxiliar utilizada y evita ordenar una lista demasiado grande. Aunque calcular las coincidencias seguirá requiriendo recorridos, estaremos evitando almacenamiento y operaciones adicionales que no necesitamos.

-Selección de fotografías

Para seleccionar las tres fotografías de amigos debemos respetar las prioridades según la cantidad de amistades en común.

Proponemos utilizar una selección aleatoria ponderada sin reemplazo. Los pesos originales de 1, 2,5 y 3,5 pueden representarse como 2, 5 y 7, manteniendo las mismas proporciones y facilitando el manejo de valores enteros.

Para evitar cálculos repetidos, podemos obtener primero los pesos de los amigos disponibles y utilizarlos durante la selección. Cada amigo elegido se excluirá de las siguientes selecciones del mismo grupo.

También tendremos que controlar que la actualización de fotografías ocurra una sola vez y contemplar qué hacer cuando el usuario no tenga suficientes amigos para mostrar tres fotografías diferentes.

-Mensajes, álbumes y fotografías

Para los mensajes proponemos conservar una sola copia de cada mensaje, asociada a su destinatario e identificando al remitente. Esto permite consultar los mensajes recibidos sin duplicar su contenido.

Como necesitamos buscar mensajes por fechas, consideramos utilizar una estructura dinámica que los mantenga ordenados cronológicamente. Un arreglo ordenado permitiría aprovechar la búsqueda binaria para localizar el inicio de un intervalo, aunque insertar mensajes podría exigir desplazar elementos. Por eso tendremos que comparar el costo de las consultas y las inserciones antes de definir la estructura final.

No incluiremos notificaciones ni una opción para consultar nuestros mensajes enviados. Para los álbumes y fotografías proponemos mantener una relación de pertenencia: cada usuario tendrá sus álbumes y cada álbum sus fotografías. No cargaremos los archivos PNG en memoria porque solamente necesitamos mostrar

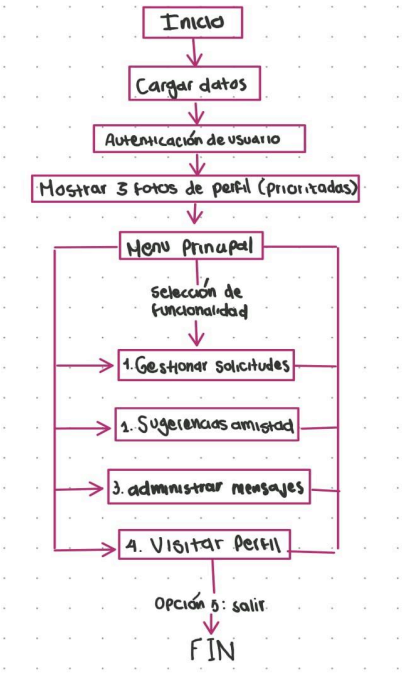
sus rutas como lo dice el desafío 2. Para el carrusel podemos utilizar una posición actual dentro de una colección con acceso directo, de forma que avanzar o retroceder no implique recorrer todas las fotografías.

-Organización y medición de recursos

La información se cargará desde los archivos y las operaciones se realizarán en memoria. Esto evita utilizar el almacenamiento permanente como espacio de procesamiento.

Utilizaremos estructuras dinámicas propias, sin contenedores STL ni struct. Además, procuraremos evitar copias innecesarias de objetos y liberar correctamente la memoria reservada.

La medición de recursos también debe considerarse desde el diseño. Necesitaremos registrar las iteraciones de cada funcionalidad, incluyendo las operaciones internas que se ejecuten, y contabilizar la memoria ocupada por los objetos, las estructuras dinámicas y las variables correspondientes. Estas mediciones nos permitirán comprobar si nuestras decisiones realmente ofrecen ventajas y no quedarnos solamente con estimaciones teóricas.

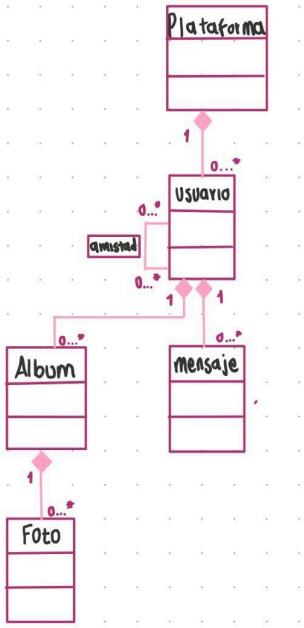


-Funcionamiento general

El programa comenzará cargando la información de los archivos. Después validará el inicio de sesión y mostrará las fotografías seleccionadas y el menú principal.

Desde el menú se administrarán amistades, sugerencias, álbumes y mensajes. Las operaciones utilizarán las estructuras en memoria y actualizarán los datos cuando corresponda. Después de ejecutar las funcionalidades se reportarán los recursos utilizados. Finalmente, al cerrar el programa, se conservarán los cambios pendientes en el almacenamiento permanente.

3. Diagrama de clases



Para el diseño inicial proponemos cinco clases: Plataforma, Usuario, Album, Foto y Mensaje.

La clase Plataforma coordinará las operaciones generales y las estructuras utilizadas para administrar los usuarios y sus relaciones. Usuario representará a las personas registradas; Álbum agrupará las fotografías de cada usuario; Foto representará los datos de cada imagen; y Mensaje permitirá manejar los mensajes recibidos.

Las relaciones de pertenencia permitirán organizar los usuarios dentro de la plataforma, los álbumes dentro de los usuarios, las fotografías dentro de los álbumes y los mensajes dentro de la información de sus destinatarios. La relación reflexiva de Usuario consigo misma permitirá representar conceptualmente las amistades entre usuarios sin crear una clase adicional para los amigos. Por ahora el diagrama será resumido, por lo que únicamente incluirá los nombres de las clases, sus relaciones y cardinalidades, sin atributos ni métodos.

Las estructuras que usamos para optimizar el funcionamiento, como la matriz de adyacencia empaquetada en bits, no necesitan convertirse en clases adicionales. Formarán parte del manejo interno del programa.